An 8-week, project-led handbook

Python for Senior C# Developers

A practical bridge from mature .NET engineering to idiomatic Python. The goal is not to relearn programming—it is to replace C# assumptions with Python's runtime model, tooling, and design habits.

WHAT TO DO

- 80 focused lessons
- 30% study · 70% implementation
- Examples, projects, checks, and authoritative links

Senior insight Build small systems early. Fluency comes from changing design instincts, not memorizing syntax.

Go deeper → Python documentation https://docs.python.org/3/

Read less. Build more.

How to use this handbook

Treat every page as a short design review. Read the concept, type the example, then apply it to the weekly project. Do not copy solutions without running them and changing one assumption.

WHAT TO DO

- Three 90-minute build sessions each week
- One 60-minute reading and review session
- Finish each week with tests and a short retrospective

```
python -m venv .venv
python -m pip install -U pip
```

Senior insight Keep one repository for the full journey so earlier modules can evolve into the final service.

Go deeper → Python tutorial https://docs.python.org/3/tutorial/

Production habits beat course completion

The 30/70 learning model

Senior engineers learn a language fastest when every feature is attached to a real constraint: malformed input, cancellation, packaging, observability, or a clean deployment.

WHAT TO DO

- 30%: documentation and focused exercises
- 50%: weekly implementation
- 20%: tests, review, and refactoring

```
# Weekly loop
learn()
build()
test()
refactor()
```

Senior insight Do not complete several overlapping beginner courses. Use one primary reference and keep shipping.

Go deeper → Exercism Python track https://exercism.org/tracks/python

Python is not C# with fewer braces

The mental-model shift

Both languages support object-oriented, functional, and asynchronous styles. The difference is where guarantees live: C# pushes many decisions to compile time; Python combines runtime protocols with optional static analysis.

WHAT TO DO

- Prefer behavior over declared ancestry
- Use types to communicate—not to imitate C#
- Expect conventions and tooling to replace some compiler enforcement

```
def total(items):
    return sum(x.price for x in items)
```

Senior insight Translate the requirement, not the original C# class diagram.

Go deeper → Python data model https://docs.python.org/3/reference/datamodel.html

Type hints do not change runtime behavior

Static typing → gradual typing

Python annotations improve editor support, documentation, and static checking. They are normally not enforced when the program runs, so boundary validation remains a separate concern.

WHAT TO DO

- Annotate public APIs and domain models
- Let local obvious types be inferred
- Run a checker such as mypy or Pyright in CI

```
def total(prices: list[float]) → float:
    return sum(prices)
```

Senior insight Do not add Java-style ceremony merely to satisfy a checker; model useful contracts.

Go deeper → Python typing https://docs.python.org/3/library/typing.html

Data objects without boilerplate

Records → dataclasses

A dataclass can generate initialization, representation, and equality methods. Frozen dataclasses express intent, but they do not make referenced mutable objects deeply immutable.

WHAT TO DO

- Use for domain values and configuration
- Choose frozen=True for value-like objects
- Validate invariants in __post_init__ when needed

```
from dataclasses import dataclass

@dataclass(frozen=True)
class Money:
    amount: int
    currency: str = 'EUR'
```

Senior insight A dataclass is a tool, not an automatic domain model. Keep behavior near the data when it belongs there.

Go deeper → dataclasses https://docs.python.org/3/library/dataclasses.html

Compatibility by behavior

Interfaces → protocols

A Protocol describes the operations a consumer needs. Structural typing means an implementation can satisfy the contract without explicitly inheriting from it—similar to compile-time duck typing.

WHAT TO DO

- Define narrow consumer-owned protocols
- Use ABCs when shared implementation or runtime membership matters
- Use plain duck typing inside small modules

```
from typing import Protocol

class Pricer(Protocol):
    def price(self, sku: str) → int:  ...
```

Senior insight Avoid an interface for every class. Introduce a protocol where substitution or testing creates real value.

Go deeper → Protocols https://typing.python.org/en/latest/spec/protocol.html

Pipeline thinking, lighter syntax

LINQ → expressions and iterators

Comprehensions handle common projection and filtering. Generator expressions stay lazy. itertools provides composable iterator building blocks for more complex flows.

WHAT TO DO

- Use comprehensions for simple transformations
- Use generators for large or streaming data
- Split pipelines when readability drops

```
totals = [o.total for o in orders if o.paid]
revenue = sum(
    o.total for o in orders
    if o.paid
)
```

Senior insight Do not force a long LINQ-style chain into one expression; name meaningful stages.

Go deeper → Functional programming tools https://docs.python.org/3/howto/functional.html

Async is still cooperative concurrency

Task → coroutine and task

Calling an async function creates a coroutine. Scheduling it as a task allows concurrent progress while operations await. One blocking call can still stall the entire event loop.

WHAT TO DO

- Use async for high-latency I/O
- Set timeouts at external boundaries
- Propagate cancellation instead of swallowing it

```
async with asyncio.TaskGroup() as tg:
    for url in urls:
        tg.create_task(fetch(url))
```

Senior insight async does not make CPU-heavy work faster. Move CPU work to a process or specialized service.

Go deeper → asyncio https://docs.python.org/3/library/asyncio.html

Lifecycle as a language protocol

using → context manager

The with statement guarantees exit logic around a block. Files, locks, database transactions, and custom resources can expose the context-manager protocol.

WHAT TO DO

- Acquire as late as possible
- Keep the protected scope small
- Create custom managers with contextlib

```
from pathlib import Path

with Path('orders.json').open() as f:
    data = f.read()
```

Senior insight Context managers govern more than disposal: they can define transaction and rollback semantics.

Go deeper → contextlib https://docs.python.org/3/library/contextlib.html

One standard project metadata file

NuGet → PyPI + pyproject.toml

pyproject.toml describes build requirements, project metadata, dependencies, and often tool configuration. Virtual environments isolate installed packages per project.

WHAT TO DO

- Use a src/ layout for reusable packages
- Keep runtime and development dependencies explicit
- Build and install in a clean environment

```
[project]
name = 'commerce-pricing'
version = '0.1.0'
requires-python = '≥3.12'
```

Senior insight A lock file and pyproject.toml solve different problems: repeatable application installs versus package metadata.

Go deeper → Packaging projects https://packaging.python.org/en/latest/tutorials/packaging-projects/

Plain assertions, powerful fixtures

xUnit → pytest

pytest discovers tests and rewrites assert statements to produce useful failures. Fixtures model reusable setup and teardown through dependency injection by function argument.

WHAT TO DO

- Test behavior, not implementation details
- Parametrize rules and boundary cases
- Keep fixtures small and explicit

```
def test_percentage_discount():
    assert discount(1000, 10) == 900
```

Senior insight A giant shared fixture graph becomes as opaque as an overgrown DI container.

Go deeper → pytest documentation https://docs.pytest.org/en/stable/

Pass collaborators where they are used

DI container → explicit injection

Python applications often inject dependencies through constructors or function parameters. Framework-managed containers are useful at application boundaries, but rarely necessary throughout the domain.

WHAT TO DO

- Inject clocks, gateways, and repositories
- Keep object construction in one composition root
- Prefer a callable when one operation is enough

```
def checkout(order, charge):
    receipt = charge(order.total)
    return receipt
```

Senior insight Dynamic typing is not a reason to hide dependencies in module globals.

Go deeper → Dependency injection in FastAPI https://fastapi.tiangolo.com/tutorial/dependencies/

Choose the framework for the product

ASP.NET Core → Python web choices

FastAPI fits typed APIs and OpenAPI-first services. Django includes ORM, authentication, admin, and a mature full-stack ecosystem. Flask stays intentionally small.

WHAT TO DO

- FastAPI: typed service APIs
- Django: integrated business applications
- Flask: small services and custom stacks

```
from fastapi import FastAPI
app = FastAPI()

@app.get('/health')
def health(): return {'ok': True}
```

Senior insight Framework choice is architecture only at the edges. Keep domain logic importable without the web framework.

Go deeper → FastAPI tutorial https://fastapi.tiangolo.com/tutorial/

Separate repeatable sources from one-pass state

IEnumerable → iterable, iterator, generator

An iterable can produce an iterator. An iterator tracks traversal state and is consumed once. A generator is a concise way to implement an iterator with suspended execution.

WHAT TO DO

- Document one-pass APIs clearly
- Return iterables instead of materialized lists when useful
- Convert to list only when callers need replay or indexing

```
def paid(orders):
    for order in orders:
        if order.paid:
            yield order
```

Senior insight Calling iter() on an iterator returns the same consumed object; this surprises code expecting replay.

Go deeper → Iterator types https://docs.python.org/3/library/stdtypes.html#iterator-types

Build an order-processing CLI

Syntax and idioms

Focus on collections, slicing, unpacking, comprehensions, truthiness, keyword arguments, exceptions, naming, and pattern matching. The deliverable reads JSON, validates orders, calculates totals, groups customers, and writes a report.

WHAT TO DO

- Keep I/O separate from pricing rules
- Use pathlib and json from the standard library
- Reject malformed records with useful context

```
match event:
    case {'type': 'paid', 'order_id': oid}:
        handle_paid(oid)
    case _:
        raise ValueError('unknown event')
```

Senior insight Success means the code feels native: few unnecessary classes, clear names, and tests around malformed input.

Go deeper → Control flow https://docs.python.org/3/tutorial/controlflow.html

Names bind to objects

Everything is an object

Variables are names, not typed storage locations. Assignment binds a name to an object; passing an argument binds another local name to that same object. This explains mutation, identity, and many debugging surprises.

WHAT TO DO

- Inspect type(obj), id(obj), and repr(obj)
- Understand rebinding versus mutation
- Do not use C# value/reference terminology mechanically

```
cart = []
other = cart
other.append('sku-1')
assert cart == ['sku-1']
```

Senior insight Reason about object identity and aliases before blaming “pass by reference.”

Go deeper → Data model: objects https://docs.python.org/3/reference/datamodel.html#objects-values-...

The object decides whether state can change

Mutability and immutability

Lists and dictionaries are mutable; tuples, strings, and integers are immutable. An immutable container may still reference mutable values, so immutability is not automatically deep.

WHAT TO DO

- Use tuples for fixed heterogeneous records sparingly
- Copy mutable input when ownership must be isolated
- Never use a mutable object as a default argument

```
def add(item, items=None):
    items = [] if items is None else items
    items.append(item)
    return items
```

Senior insight Mutable defaults are created once at function definition time, not once per call.

Go deeper → Python FAQ: shared defaults https://docs.python.org/3/faq/programming.html#why-are-default-v...

is answers a different question than ==

Identity versus equality

Equality asks whether values compare equal. Identity asks whether two names reference the exact same object. Use is for None and deliberate singleton sentinels—not for strings or numbers.

WHAT TO DO

- Write value equality through dataclasses or __eq__
- Use `value is None`
- Create a unique sentinel with object()

```
MISSING = object()
value = payload.get('coupon', MISSING)
if value is MISSING:
    handle_absent_coupon()
```

Senior insight Interning can make identity tests appear to work until they fail in another runtime or execution.

Go deeper → Comparisons https://docs.python.org/3/reference/expressions.html#comparisons

Copying is an ownership decision

Shallow and deep copies

A shallow copy creates a new outer container but reuses nested references. deepcopy recursively duplicates supported objects, which may be expensive or semantically wrong for resources and shared identities.

WHAT TO DO

- Prefer constructing a new domain value explicitly
- Use copy.copy for a new outer container
- Reserve deepcopy for genuinely tree-like state

```
from copy import copy
updated = copy(order)
updated.lines = [*order.lines, new_line]
```

Senior insight Blind deepcopy can duplicate state that should remain shared or cannot be copied safely.

Go deeper → copy module https://docs.python.org/3/library/copy.html

Generated methods with explicit trade-offs

Dataclasses in depth

Dataclasses can generate init, repr, equality, ordering, slots, and frozen behavior. default_factory creates a fresh mutable value for every instance.

WHAT TO DO

- Use slots=True after measuring or for strong shape constraints
- Use field(default_factory=list)
- Keep validation and derived values explicit

```
from dataclasses import dataclass, field
@dataclass(slots=True)
class Cart:
    lines: list[str] = field(default_factory=list)
```

Senior insight unsafe_hash=True deserves scrutiny; hashing mutable business state can corrupt dictionaries and sets.

Go deeper → dataclasses reference https://docs.python.org/3/library/dataclasses.html

Preserve a simple attribute-facing API

Properties

A property allows computed access or validation while callers use attribute syntax. It is useful when behavior belongs to the object, but not as a reason to port every C# property verbatim.

WHAT TO DO

- Expose plain attributes by default
- Add a property when computation or invariant requires it
- Avoid hidden network or database I/O in properties

```
class Order:
    @property
    def total(self) → int:
        return sum(x.total for x in self.lines)
```

Senior insight A property should be unsurprising and cheap; expensive work deserves an explicit method.

Go deeper → property built-in https://docs.python.org/3/library/functions.html#property

Integrate with Python's data model

Dunder methods

Special methods let your type participate in built-in syntax and protocols: repr, equality, iteration, length, context management, and more. Implement only semantics users can predict.

WHAT TO DO

- Make __repr__ useful for debugging
- Return NotImplemented for unsupported comparisons
- Prefer iteration over custom collection APIs

```
class Cart:
    def __iter__(self):
        return iter(self.lines)
    def __len__(self):
        return len(self.lines)
```

Senior insight Do not call special methods directly when a built-in such as len() or iter() expresses the protocol.

Go deeper → Special method names https://docs.python.org/3/reference/datamodel.html#special-metho…

Small collaborators, explicit behavior

Composition over inheritance

Python’s multiple inheritance and dynamic model are powerful, but application code is usually clearer when behavior is composed from focused objects or functions.

WHAT TO DO

- Inject a pricing policy instead of subclassing Order
- Use inheritance for genuine substitutability
- Keep mixins narrow and behavior-focused

```
class Checkout:
    def __init__(self, price):
        self.price = price
    def run(self, cart):
        return self.price(cart)
```

Senior insight A function is often the smallest useful strategy object in Python.

Go deeper → Classes tutorial https://docs.python.org/3/tutorial/classes.html

Model behavior without enterprise ceremony

Project: pricing library

Build products, Money, carts, discounts, and pluggable pricing policies. Make the package usable from a CLI and from tests without framework dependencies.

WHAT TO DO

- Represent money in minor units or Decimal
- Define rounding deliberately
- Test competing discount rules and empty carts

```
def percentage(percent: int):
    def apply(total: int) → int:
        return total * (100 - percent) // 100
    return apply
```

Senior insight Completion means readable domain code, useful reprs, and no "IProductServiceFactoryManager."

Go deeper → decimal module https://docs.python.org/3/library/decimal.html

Types are executable metadata

Function and variable annotations

Annotations live on functions and classes and can be inspected by tools. Postponed evaluation and forward references affect runtime inspection, so frameworks may resolve hints explicitly.

WHAT TO DO

- Annotate module boundaries first
- Use collections.abc for behavioral inputs
- Return precise types instead of Any

```
from collections.abc import Iterable

def revenue(values: Iterable[int]) → int:
    return sum(values)
```

Senior insight A type checker cannot rescue an unclear API. Improve the model before adding complex type machinery.

Go deeper → Typing best practices https://typing.python.org/en/latest/reference/best_practices.html

Model absence deliberately

Containers, unions, and optional values

Modern annotations use built-in generics and the | union operator. T | None means absence is valid; it does not supply a default and should be narrowed before use.

WHAT TO DO

- Prefer specific containers at boundaries
- Accept Sequence or Iterable when mutation is unnecessary
- Avoid large unions that hide multiple responsibilities

```
def find_order(order_id: str) -> Order | None:
    ...

if (order := find_order('A1')) is not None:
    print(order.total)
```

Senior insight Optional is not a synonym for “parameter has a default.”

Go deeper → Type annotation syntax https://docs.python.org/3/library/typing.html

Preserve relationships between types

Protocols and generics

TypeVar and generic syntax describe how input and output types relate. Protocol captures required behavior. Use both when they improve a consumer's understanding—not to reproduce .NET abstraction layers.

WHAT TO DO

- Keep protocols minimal
- Bind type variables when operations require capabilities
- Prefer concrete types for internal implementation details

```
from typing import Protocol, TypeVar
T = TypeVar('T')
class Repo(Protocol[T]):
    def get(self, key: str) → T | None:  ...
```

Senior insight If a generic has no meaningful type relationship, a non-generic API is usually clearer.

Go deeper → Generics https://typing.python.org/en/latest/reference/generics.html

Functions are first-class dependencies

Callable and higher-order design

Callable describes a function-like collaborator. Closures can capture configuration and return a focused operation, replacing small one-method classes when identity and lifecycle are unimportant.

WHAT TO DO

- Type callable parameters and return values
- Use functools.partial for simple binding
- Choose a class when state or multiple operations belong together

```
from collections.abc import Callable
Price = Callable[[Cart], int]

def checkout(cart: Cart, price: Price) → int:
    return price(cart)
```

Senior insight Callables make excellent test seams, but give important domain operations meaningful names.

Go deeper → Callable types https://docs.python.org/3/library/typing.html#annotating-callable-o…

Prove which branch you are in

Type narrowing

Checkers narrow unions after isinstance, None checks, match statements, and user-defined TypeGuard or TypeIs functions. Narrow at boundaries, then keep the core model precise.

WHAT TO DO

- Validate untrusted input before narrowing
- Use assert only for programmer invariants
- Avoid cast unless runtime evidence exists elsewhere

```
def total(value: int | str) → int:
    if isinstance(value, str):
        value = int(value)
    return value
```

Senior insight cast changes the checker's opinion; it performs no runtime conversion or validation.

Go deeper → Type narrowing https://typing.python.org/en/latest/guides/type_narrowing.html

Design a stable public surface

Packages, modules, and imports

A module is a Python file; a package organizes modules. Imports execute module top-level code once per interpreter process and cache the module object.

WHAT TO DO

- Keep import-time work cheap
- Expose a small public API from __init__.py
- Break circular imports by fixing dependency direction

```
src/commerce_pricing/
  __init__.py
  models.py
  pricing.py
tests/
```

Senior insight An import that opens connections or reads environment state makes tests and tooling fragile.

Go deeper → Modules tutorial https://docs.python.org/3/tutorial/modules.html

Isolate installations per project

Virtual environments

A virtual environment provides a project-specific interpreter context and site-packages directory. Activation is convenience; calling the environment's interpreter directly is equally valid.

WHAT TO DO

- Create .venv outside source control
- Use python -m pip to target the active interpreter
- Recreate environments instead of repairing mystery state

```
python -m venv .venv
# Windows
.venv\Scripts\activate
python -m pip install -e .
```

Senior insight The environment is disposable; project metadata and lock data are the reproducible source.

Go deeper → Virtual environments https://docs.python.org/3/tutorial/venv.html

Make the library installable

pyproject.toml and package project

Move the pricing library under src/, declare metadata and dependencies, and add tool configuration. Verify editable installation during development and a built wheel in a clean environment.

WHAT TO DO

- Build both wheel and source distribution
- Test the installed artifact—not only the checkout
- Document supported Python versions

```
python -m pip install -e '.[dev]'
python -m build
python -m pytest
```

Senior insight A green test suite in the repository does not prove the package contains all required files.

Go deeper → pyproject.toml specification https://packaging.python.org/en/latest/specifications/pyproject-toml/

Tests are ordinary functions

pytest fundamentals

pytest discovers tests, rewrites assertions, captures output, and reports rich diffs. Arrange behavior around public APIs rather than mirroring every implementation function.

WHAT TO DO

- Name tests after behavior and condition
- Use plain assert expressions
- Keep a fast default test command

```
def test_empty_cart_costs_zero():
    assert price(Cart()) == 0
```

Senior insight One clear assertion story matters more than enforcing one assert per test.

Go deeper → pytest get started https://docs.pytest.org/en/stable/getting-started.html

Reusable setup with scoped lifetime

Fixtures

Fixtures are requested by name and can depend on other fixtures. yield fixtures provide teardown. Scope controls reuse across functions, classes, modules, packages, or sessions.

WHAT TO DO

- Default to function scope
- Prefer data builders over giant object graphs
- Keep teardown next to acquisition

```
@pytest.fixture
def cart():
    c = Cart()
    c.add(Product('A', 500))
    return c
```

Senior insight If a test's inputs are invisible inside distant fixtures, readability has been traded for reuse.

Go deeper → pytest fixtures https://docs.pytest.org/en/stable/how-to/fixtures.html

Turn rule tables into executable specifications

Parametrization

Parametrized tests run one test body against many cases and report each case independently. Give domain cases readable IDs so failures explain the business rule.

WHAT TO DO

- Use for boundaries and decision tables
- Keep expected values explicit
- Do not hide a second algorithm in the test

```
@pytest.mark.parametrize('total,pct,expected',[
    (1000, 0, 1000),
    (1000, 10, 900),
])
def test_discount(total,pct,expected):
    assert discount(total,pct) == expected
```

Senior insight Large parameter matrices may indicate the production API has too many independent dimensions.

Go deeper → Parametrize tests https://docs.pytest.org/en/stable/how-to/parametrize.html

Replace boundaries, not internals

Test doubles and monkeypatch

monkeypatch temporarily changes attributes, environment variables, dictionaries, or paths. Prefer injecting a gateway or callable; patch only where the code under test looks up the dependency.

WHAT TO DO

- Fake HTTP at the gateway boundary
- Use tmp_path for file behavior
- Assert outcomes before mock call trivia

```
def test_checkout_decline():
    def decline(_): raise CardDeclined()
    with pytest.raises(CardDeclined):
        checkout(cart, decline)
```

Senior insight Over-mocking locks tests to implementation and makes refactoring expensive.

Go deeper → monkeypatch https://docs.pytest.org/en/stable/how-to/monkeypatch.html

Test failure semantics explicitly

Exceptions and coverage

Use pytest.raises to verify exception type and meaningful details. Coverage shows which code ran; branch coverage is more useful for decision-heavy logic, but neither measures test quality.

WHAT TO DO

- Test recovery and cleanup paths
- Include context in domain exceptions
- Review uncovered branches, not just percentages

```
with pytest.raises(
    InvalidCoupon,
    match='expired',
):
    apply_coupon(cart, coupon)
```

Senior insight A high coverage number can coexist with assertions that prove almost nothing.

Go deeper → Coverage.py https://coverage.readthedocs.io/

Fast feedback for style and defects

Ruff

Ruff combines formatting with a broad lint rule set. Configure it centrally in pyproject.toml and adopt rules intentionally rather than enabling everything at once.

WHAT TO DO

- Run format and check in CI
- Keep ignores narrow and documented
- Use automatic fixes, then review the diff

```
ruff format .
ruff check .
ruff check . --fix
```

Senior insight A formatter ends style debates; a linter should catch risks, not become a puzzle game.

Go deeper → Ruff documentation https://docs.astral.sh/ruff/

Static checks as a quality gate

mypy or Pyright

Type checkers analyze annotations without executing the program. Start strict at new package boundaries, then ratchet older code forward instead of flooding the project with Any and ignores.

WHAT TO DO

- Check tests as well as production code
- Treat Any as an escape hatch
- Prefer narrow ignores with error codes

```
mypy src tests
# or
pyright
```

Senior insight Passing static checks does not validate JSON, HTTP responses, or database rows at runtime.

Go deeper → mypy documentation https://mypy.readthedocs.io/en/stable/

Record events for operators

Logging

Use the logging module or a structured layer to emit levels, context, and exceptions. Configure handlers at the application boundary; libraries should not decide global output destinations.

WHAT TO DO

- Log identifiers, duration, and outcome
- Avoid secrets and personal data
- Use logger.exception inside exception handling

```
logger.info('order_submitted', extra={
    'order_id': order.id, 'total': order.total
})
```

Senior insight Logs are an interface for diagnosis. Stable event names beat prose that changes every refactor.

Go deeper → Logging HOWTO https://docs.python.org/3/howto/logging.html

Make quality executable

Project: professional test suite

Add unit tests for pricing, integration tests for package boundaries, linting, formatting, type checking, and coverage reporting. One documented command should reproduce the local gate.

WHAT TO DO

- Keep unit tests network-free
- Test installation in CI
- Fail on warnings you own

```
python -m pytest
ruff format --check .
ruff check .
mypy src tests
```

Senior insight The deliverable is not “many tests”; it is fast, trustworthy feedback around risky behavior.

Go deeper → pytest good practices https://docs.pytest.org/en/stable/explanation/goodpractices.html

A protocol with stateful traversal

Iterable versus iterator

iter(x) asks for an iterator; next(it) advances it until StopIteration. Collections usually create fresh iterators, while iterator objects are commonly one-shot.

WHAT TO DO

- Accept Iterable when replay is unnecessary
- Return Iterator for one-pass results
- Do not call len() on arbitrary iterables

```
it = iter(['A','B'])
assert next(it) == 'A'
assert next(it) == 'B'
```

Senior insight Consuming an iterator for logging or validation may leave nothing for the real consumer.

Go deeper → Iterator protocol https://docs.python.org/3/library/stdtypes.html#iterator-types

Write state machines as straight-line code

Generators and yield

A generator function returns a generator object. Each yield produces a value and suspends local state until the next request, enabling clear incremental pipelines.

WHAT TO DO

- Use yield from to delegate iteration
- Put cleanup in try/finally when needed
- Keep generator side effects explicit

```
def valid(rows):
    for row in rows:
        if row.get('sku'):
            yield row
```

Senior insight Exceptions may occur during iteration, not when the generator is created—test the consumption point.

Go deeper → Generator expressions https://docs.python.org/3/reference/expressions.html#yield-expressi…

Pay only for consumed values

Lazy evaluation

Generator expressions compose filtering and projection without allocating intermediate lists. Laziness improves memory use and early termination, but defers work and failures.

WHAT TO DO

- Use any, all, sum, min, and max as consumers
- Stop early with next or itertools.islice
- Materialize at stable boundaries when debugging or replay matters

```
first_large = next(
    (o for o in orders if o.total > 10_000),
    None,
)
```

Senior insight A lazy pipeline tied to an open file or database cursor must not outlive its resource scope.

Go deeper → Generator expressions https://docs.python.org/3/reference/expressions.html#generator-exp…

Composable iterator building blocks

itertools

itertools supplies efficient primitives such as chain, islice, groupby, pairwise, and batching. Many operate lazily and assume specific input ordering.

WHAT TO DO

- Sort before groupby when grouping globally
- Use islice for bounded previews
- Use batched for fixed-size chunks

```
from itertools import batched
for chunk in batched(records, 100):
    write_batch(chunk)
```

Senior insight groupby groups adjacent equal keys; it is not LINQ GroupBy over unsorted input.

Go deeper → itertools https://docs.python.org/3/library/itertools.html

Wrap callables without changing callers

Decorators

A decorator receives a callable and returns a replacement. functools.wraps preserves metadata that frameworks, debuggers, and documentation tools rely on.

WHAT TO DO

- Use for cross-cutting callable behavior
- Keep order of stacked decorators visible
- Prefer explicit code when wrapping obscures control flow

```
from functools import wraps
def traced(fn):
    @wraps(fn)
    def wrapper(*a, **k):
        return fn(*a, **k)
    return wrapper
```

Senior insight Decorators can hide I/O, retries, or transactions; make consequential behavior obvious in naming and docs.

Go deeper → Function decorators https://docs.python.org/3/glossary.html#term-decorator

Make acquire/use/release atomic in code

Context managers

Classes implement __enter__/__exit__; contextlib.contextmanager turns a generator into a manager. Exit logic runs whether the block succeeds or raises.

WHAT TO DO

- Use ExitStack for dynamic resource sets
- Do not suppress exceptions accidentally
- Make transaction boundaries visible

```
from contextlib import contextmanager
@contextmanager
def transaction(db):
    tx = db.begin()
    try: yield tx
    except: tx.rollback(); raise
    else: tx.commit()
```

Senior insight Returning True from __exit__ suppresses an exception; do it only as a deliberate API promise.

Go deeper → Context manager types https://docs.python.org/3/library/contextlib.html

Compose functions, process bounded memory

functools and streaming files

partial binds arguments; cache memoizes deterministic calls; singledispatch provides type-based function variants. Combine these carefully with line-by-line file iteration.

WHAT TO DO

- Use pathlib for paths
- Specify text encoding explicitly
- Cache only bounded, stable key spaces

```
from pathlib import Path
with Path('sales.csv').open(encoding='utf-8') as f:
    for line in f:
        process(line)
```

Senior insight An unbounded cache is a memory leak with a friendly API.

Go deeper → functools https://docs.python.org/3/library/functools.html

Process data without loading it all

Project: lazy sales pipeline

Read CSV transactions, validate and normalize rows, convert currencies through an injected service, aggregate results, and write rejected rows separately.

WHAT TO DO

- Stream input and output
- Separate parse, validate, enrich, and aggregate stages
- Report line numbers with errors

```
rows = read_csv(path)
clean = (validate(r) for r in rows)
converted = (convert(r, rates) for r in clean)
report(converted)
```

Senior insight Measure memory with a realistically large fixture and prove the pipeline begins producing output early.

Go deeper → csv module https://docs.python.org/3/library/csv.html

Threading is useful—but not universal parallelism

The GIL: practical model

In standard CPython, a global interpreter lock coordinates execution of Python bytecode. Threads still overlap blocking I/O effectively; CPU-bound Python usually needs processes, native code that releases the lock, or another architecture.

WHAT TO DO

- Benchmark the real workload
- Use threads for blocking libraries
- Use processes for isolated CPU work

```
# Choose by bottleneck
I/O bound  → threads or asyncio
CPU bound  → processes
```

Senior insight Do not use the GIL as a blanket reason to avoid threads; use it to choose them accurately.

Go deeper → Threading and the GIL https://docs.python.org/3/library/threading.html#gil-and-performan…

Concurrent calls with familiar code

Threads for blocking I/O

ThreadPoolExecutor runs ordinary callables in worker threads and returns futures. Bound the pool, propagate failures, and make shared state minimal.

WHAT TO DO

- Suitable for blocking SDKs and file I/O
- Use locks around shared mutable invariants
- Shut executors down with a context manager

```
with ThreadPoolExecutor(max_workers=8) as pool:
    prices = list(pool.map(fetch_price, skus))
```

Senior insight More workers can reduce throughput by overwhelming dependencies or increasing contention.

Go deeper → concurrent.futures https://docs.python.org/3/library/concurrent.futures.html

Parallelism with serialization costs

Processes for CPU work

ProcessPoolExecutor uses separate processes. Arguments and results cross a serialization boundary, so functions and data must be transferable and startup overhead must be justified.

WHAT TO DO

- Batch small work units
- Protect process entry points
- Avoid sending huge object graphs repeatedly

```
if __name__ == '__main__':
    with ProcessPoolExecutor() as pool:
        results = list(pool.map(score, batches))
```

Senior insight A process pool is not a free speed switch; measure serialization, startup, and memory multiplication.

Go deeper → ProcessPoolExecutor https://docs.python.org/3/library/concurrent.futures.html#processpo…

One event loop, cooperative progress

asyncio fundamentals

Async code yields control at await points while waiting for compatible operations. The event loop schedules ready tasks; regular blocking calls prevent every other task from progressing.

WHAT TO DO

- Keep async from API boundary to I/O client
- Use asyncio.run at the program entry point
- Never sprinkle await without an async-capable dependency

```
async def main():
    result = await fetch('sku-1')
    print(result)

asyncio.run(main())
```

Senior insight Async improves concurrency and resource use; it does not automatically reduce single-request latency.

Go deeper → asyncio conceptual overview https://docs.python.org/3/library/asyncio.html

Creation is not scheduling

Coroutines and tasks

Calling an async function returns a coroutine object. Awaiting runs it as part of the current task; create_task schedules independent progress and requires lifecycle ownership.

WHAT TO DO

- Keep strong references to background tasks
- Await tasks before shutdown
- Use TaskGroup for related child work

```
async with asyncio.TaskGroup() as tg:
    tasks = [tg.create_task(fetch(s)) for s in skus]
prices = [t.result() for t in tasks]
```

Senior insight Fire-and-forget tasks can disappear, fail silently, or outlive resources.

Go deeper → Coroutines and tasks https://docs.python.org/3/library/asyncio-task.html

Latency budgets are part of correctness

Cancellation and timeouts

asyncio.timeout bounds an awaitable region. Cancellation is delivered as CancelledError; cleanup should run, then cancellation should usually propagate.

WHAT TO DO

- Set connect and operation timeouts
- Budget nested calls from the outer deadline
- Clean up in finally or async context managers

```
async with asyncio.timeout(2.0):
    return await provider.price(sku)
```

Senior insight Catching BaseException or CancelledError without re-raising can make shutdown hang.

Go deeper → Timeouts https://docs.python.org/3/library/asyncio-task.html#timeouts

Child tasks belong to a scope

Structured concurrency

TaskGroup waits for all children and coordinates failure. If one task fails, siblings are cancelled and errors are reported together, making lifetime and failure semantics explicit.

WHAT TO DO

- Group work that succeeds or fails together
- Handle ExceptionGroup selectively
- Keep independent workflows in separate groups

```
try:
    async with asyncio.TaskGroup() as tg:
        for p in providers:
            tg.create_task(p.fetch(sku))
except* ProviderError as errors:
    record(errors)
```

Senior insight Structured concurrency prevents orphan tasks; it does not define your business policy for partial results.

Go deeper → Task groups https://docs.python.org/3/library/asyncio-task.html#task-groups

Await resource acquisition and cleanup

Async context managers

async with supports resources whose enter or exit work is asynchronous: HTTP sessions, database connections, locks, and streaming responses.

WHAT TO DO

- Create clients once per application scope when supported
- Close resources during lifespan shutdown
- Keep transaction scopes short

```
async with client.stream('GET', url) as response:
    async for chunk in response.aiter_bytes():
        consume(chunk)
```

Senior insight Creating a new network client per request can defeat connection pooling.

Go deeper → Asynchronous context managers https://docs.python.org/3/reference/datamodel.html#asynchronous-...

One accidental call stalls all tasks

Avoid blocking the event loop

File APIs, synchronous HTTP clients, CPU-heavy loops, and time.sleep block the event-loop thread. Replace them with async libraries, asyncio.to_thread, or process-based work.

WHAT TO DO

- Use asyncio.sleep, not time.sleep
- Profile slow callbacks
- Wrap legacy blocking calls deliberately

```
result = await asyncio.to_thread(
    legacy_sdk.call, sku
)
```

Senior insight to_thread protects responsiveness but does not turn unsafe libraries into thread-safe ones.

Go deeper → Running blocking functions https://docs.python.org/3/library/asyncio-task.html#running-in-threa…

Concurrency with bounded failure

Project: price checker

Query several product providers concurrently, limit in-flight calls, apply per-provider timeouts, and return available prices with explicit errors.

WHAT TO DO

- Use a semaphore or client connection limits
- Preserve provider identity in results
- Test timeout, cancellation, and partial failure

```
limit = asyncio.Semaphore(10)
async def bounded(provider, sku):
    async with limit:
        return await provider.price(sku)
```

Senior insight The fastest implementation is not one that melts the upstream service.

Go deeper → Synchronization primitives https://docs.python.org/3/library/asyncio-sync.html

Make HTTP contracts explicit

FastAPI models

FastAPI uses type hints and validation models to parse inputs, document OpenAPI, and serialize outputs. Use separate request, response, and domain types when their responsibilities differ.

WHAT TO DO

- Declare status codes and response models
- Keep domain behavior framework-independent
- Version contracts deliberately

```
class CreateOrder(BaseModel):
    customer_id: str
    lines: list[OrderLine]

@app.post('/orders', status_code=201)
def create(command: CreateOrder):  ...
```

Senior insight Do not return ORM entities directly; persistence shape is not a public API contract.

Go deeper → Request body models https://fastapi.tiangolo.com/tutorial/body/

Composition at the web boundary

Function-based dependencies

Depends resolves callables and their sub-dependencies. Use it for authentication, request-scoped sessions, and adapters—not to hide every ordinary function call.

WHAT TO DO

- Cache application singletons explicitly
- Yield dependencies for cleanup
- Override dependencies in tests

```
def get_service(repo=Depends(get_repo)):
    return OrderService(repo)

@app.post('/orders')
def create(
    cmd: CreateOrder,
    svc=Depends(get_service),
):
    return svc.create(cmd)
```

Senior insight A dependency graph should remain understandable without framework magic.

Go deeper → FastAPI dependencies https://fastapi.tiangolo.com/tutorial/dependencies/

Translate boundary failures consistently

Validation and error handling

Input models reject malformed requests before handlers run. Domain exceptions should be translated into stable HTTP responses through exception handlers rather than leaking stack details.

WHAT TO DO

- Differentiate invalid input, conflict, and absence
- Return machine-readable error codes
- Log internal context without exposing secrets

```
@app.exception_handler(OrderNotFound)
async def missing(_, exc):
    return JSONResponse(
        status_code=404,
        content={
            'code': 'order_not_found'
        },
    )
```

Senior insight Do not return ORM entities directly; persistence shape is not a public API contract.

Go deeper → Handling errors https://fastapi.tiangolo.com/tutorial/handling-errors/

Own cross-cutting request and app lifecycle

Middleware and lifespan

Middleware wraps every request for concerns such as correlation IDs and timing. Lifespan manages startup and shutdown resources such as pools and shared clients.

WHAT TO DO

- Keep middleware fast and non-blocking
- Initialize shared resources once
- Close resources deterministically

```
@asynccontextmanager
async def lifespan(app):
    app.state.client = AsyncClient()
    try:
        yield
    finally:
        await app.state.client.aclose()
```

Senior insight Do not perform slow external health checks on every request through middleware.

Go deeper → FastAPI lifespan https://fastapi.tiangolo.com/advanced/events/

Validate settings once, isolate persistence

Configuration and database boundaries

Load configuration from environment or secret stores at startup and validate it. Repositories translate between persistence records and domain objects; transaction scope belongs to the use case.

WHAT TO DO

- Never commit production secrets
- Keep sessions request-scoped
- Make transaction ownership visible

```
class Settings(BaseSettings):
    database_url: SecretStr
    model_config = SettingsConfigDict(
        env_prefix='APP_'
    )
```

Senior insight Environment variables are delivery mechanisms, not validation or secret rotation strategies.

Go deeper → Pydantic settings https://docs.pydantic.dev/latest/concepts/pydantic_settings/

Integrate without coupling the core

Project: Order Management API

Implement create, retrieve, add-line, price, and submit operations. Publish a simulated event only after the state change succeeds, with a testable boundary for delivery.

WHAT TO DO

- Generate OpenAPI from explicit contracts
- Test use cases without HTTP
- Add API tests for mapping and status codes

```
POST /orders
POST /orders/{id}/lines
POST /orders/{id}/submit
GET  /orders/{id}
```

Senior insight Completion means the domain package can run in a unit test without importing FastAPI.

Go deeper → Testing FastAPI https://fastapi.tiangolo.com/tutorial/testing/

Product Intelligence Service

Capstone architecture

Build a service that queries providers, normalizes offers, caches safe results, and exposes a typed API. Keep transport, orchestration, domain decisions, and infrastructure in separate dependency directions.

WHAT TO DO

- API → application use case → domain
- Adapters implement provider and cache protocols
- Observability surrounds every external boundary

```
app/ api/ application/ domain/ adapters/
tests/ unit/ integration/
```

Senior insight Architecture is successful when business rules remain testable without network, database, or framework startup.

Go deeper → Packaging namespace guidance https://packaging.python.org/en/latest/guides/packaging-namespac…

Thin endpoints, explicit use cases

Service boundary

HTTP handlers authenticate, validate, invoke one application operation, and translate the result. Orchestration belongs in a use-case service that is callable from HTTP, jobs, or tests.

WHAT TO DO

- Keep framework models at the edge
- Pass a request-scoped context explicitly
- Return domain results, then map to responses

```
@app.get('/products/{sku}/offers')
async def offers(
    sku: str,
    svc=Depends(service),
):
    return await svc.find_offers(sku)
```

Senior insight A thin endpoint is not an anemic domain; it simply puts decisions in reusable code.

Go deeper → Bigger FastAPI applications https://fastapi.tiangolo.com/tutorial/bigger-applications/

Normalize unreliable external systems

Async provider integrations

Each provider adapter owns authentication, request mapping, response parsing, timeout defaults, and error translation. The application consumes one stable protocol.

WHAT TO DO

- Use one shared async client per process
- Validate responses before domain mapping
- Capture provider latency and outcome

```
class Provider(Protocol):
    async def offers(
        self, sku: str
    ) → list[Offer]: ...
```

Senior insight Do not let provider-specific JSON dictionaries escape into the domain.

Go deeper → HTTPX async support https://www.python-httpx.org/async/

Represent valid states clearly

Typed domain model

Use dataclasses or focused classes for Money, Offer, ProductId, and decisions. Keep unvalidated dictionaries at boundaries and convert them to domain values once.

WHAT TO DO

- Encode currency and units
- Reject invalid values at construction
- Prefer immutable values for identifiers and money

```
@dataclass(frozen=True, slots=True)
class Offer:
    provider: str
    price: Decimal
    currency: str
```

Senior insight Type hints document shape; constructors and boundary validators enforce runtime invariants.

Go deeper → Decimal https://docs.python.org/3/library/decimal.html

Cache semantics before technology

Caching

Choose keys, freshness, invalidation, negative caching, and failure behavior before selecting an in-memory or distributed cache. Never let a stale cache silently violate correctness.

WHAT TO DO

- Include locale and currency in keys
- Add bounded TTL with jitter where appropriate
- Measure hits, misses, and load latency

```
key = f'offers:{sku}:{currency}'
if cached := await cache.get(key):
    return decode(cached)
```

Senior insight Caching is duplicated state. Every cache needs an explicit answer for when that duplication becomes invalid.

Go deeper → functools cache tools https://docs.python.org/3/library/functools.html#functools.lru_cache

Bound every remote call

Retries and timeouts

Timeouts cap waiting; retries handle selected transient failures. Use exponential backoff with jitter, cap attempts, and respect the caller's total deadline.

WHAT TO DO

- Retry idempotent operations by default
- Do not retry validation or authentication failures
- Record attempts and final outcome

```
for attempt in range(3):
    try: return await call()
    except TransientError:
        await asyncio.sleep(backoff(attempt))
```

Senior insight Retries multiply traffic during incidents. A retry policy without a budget can worsen the outage.

Go deeper → Tenacity documentation https://tenacity.readthedocs.io/

Choose durability intentionally

Background processing

In-process tasks suit short, non-critical work tied to the application lifetime. Durable jobs need a queue, persisted state, retries, idempotency, and workers independent of the web process.

WHAT TO DO

- Return 202 for accepted asynchronous work
- Generate idempotency keys
- Expose job status and failure reason

```
@app.post('/refresh/{sku}', status_code=202)
def refresh(sku: str, tasks: BackgroundTasks):
    tasks.add_task(refresh_one, sku)
```

Senior insight FastAPI BackgroundTasks is not a durable queue; process termination can lose work.

Go deeper → Background tasks https://fastapi.tiangolo.com/tutorial/background-tasks/

Connect a request across boundaries

Structured logging and observability

Emit consistent events with correlation ID, operation, provider, duration, and outcome. Add metrics for rates and latency; add traces when cross-service causality matters.

WHAT TO DO

- Use stable event names
- Propagate trace context to providers
- Keep high-cardinality values out of metric labels

```
logger.info(
 'provider_completed',
 extra={
  'provider': name,
  'duration_ms': elapsed,
  'outcome': 'ok',
 },
)
```

Senior insight Observability begins with questions operators must answer, not with installing three agents.

Go deeper → OpenTelemetry Python https://opentelemetry.io/docs/languages/python/

Test contracts at the cheapest trustworthy layer

Unit and integration strategy

Unit tests cover domain rules and orchestration with fakes. Integration tests exercise HTTP mapping, persistence, serialization, and provider adapters against controlled dependencies.

WHAT TO DO

- Use contract fixtures for provider responses
- Keep live-provider smoke tests separate
- Test cancellation and timeout behavior

```
tests/
  unit/test_ranking.py
  integration/test_api.py
  integration/test_provider_adapter.py
```

Senior insight The test pyramid is a feedback design: fast broad confidence below, targeted realism above.

Go deeper → pytest good practices https://docs.pytest.org/en/stable/explanation/goodpractices.html

Build a small, reproducible runtime

Container image

Use a supported Python base, install dependencies before frequently changing source for layer reuse, run as a non-root user, and define a clear command and health behavior.

WHAT TO DO

- Pin the base image deliberately
- Exclude caches, tests, and secrets from build context
- Install and run the built package

```
FROM python:3.14-slim
WORKDIR /app
COPY . .
RUN python -m pip install .
USER 10001
CMD ["uvicorn","app.main:app"]
```

Senior insight A container is packaging, not isolation from bad configuration or vulnerable dependencies.

Go deeper → Docker Python guide https://docs.docker.com/guides/python/

Make the clean path automatic

CI quality gates

A pull request should recreate the environment, install the project, check formatting, lint, type-check, test, build the distribution, and optionally scan dependencies and images.

WHAT TO DO

- Cache downloads, not correctness state
- Run the same commands locally and in CI
- Upload test and coverage reports on failure

```
ruff format --check .
ruff check .
mypy src tests
pytest
python -m build
```

Senior insight CI that tests only the checkout can miss broken packaging; install the artifact in at least one job.

Go deeper → GitHub Actions Python https://docs.github.com/en/actions/automating-builds-and-tests/bui…

Definition of done

Final quality checklist

The capstone is complete when another engineer can clone it, reproduce the environment, run every check, understand failures, and operate the service without hidden local state.

WHAT TO DO

- Tests, formatter, linter, and type checker pass
- Dependencies and configuration are explicit
- Timeouts, errors, logs, README, and clean setup are verified

```
python -m venv .venv
python -m pip install -e '.[dev]'
pytest && ruff check . && mypy src
```

Senior insight Do the final verification from a fresh directory or container—not from the environment that accumulated during development.

Go deeper → Python Packaging User Guide https://packaging.python.org/

Six to eight focused hours

Weekly study rhythm

Use short loops that alternate understanding and implementation. A senior engineer’s advantage is judgment; preserve it by reviewing trade-offs rather than racing through syntax.

WHAT TO DO

- 60 min: docs and C# comparison
- 2 × 90 min: project implementation
- 60 min: tests/refactoring · 30 min: exercises · 30 min: review

```
Mon: concept + spike
Wed: feature + tests
Sat: integration + retrospective
```

Senior insight Write one sentence each week: “The Python approach differs because…” This makes mental-model changes visible.

Go deeper → Python tutorial https://docs.python.org/3/tutorial/

Do not rebuild .NET by hand

What C# developers should avoid

The common failure mode is syntactically valid Python wrapped in unnecessary factories, interfaces, DTO layers, and service locators. Keep boundaries strong while allowing local code to stay simple.

WHAT TO DO

- No getter/setter boilerplate by default
- No interface for every collaborator
- No async for CPU work; no broad exception swallowing

```
# Prefer this
def price(cart, policy): return policy(cart)
# over a hierarchy with one method per type
```

Senior insight Pythonic does not mean unstructured. It means choosing the lightest mechanism that preserves clarity and correctness.

Go deeper → Python design FAQ https://docs.python.org/3/faq/design.html

Apply the core to your destination

Choose a specialization

After the eight-week core, deepen one path while maintaining the same engineering gates. Build one portfolio artifact that demonstrates decisions, tests, and operation—not just framework syntax.

WHAT TO DO

- Backend: FastAPI, SQL, queues, observability
- AI automation: model APIs, tools, evals, safety, workflows
- Data: NumPy, pandas/Polars, notebooks, pipelines

```
core = {
 'typing', 'testing',
 'packaging', 'async',
 'quality',
}
path = choose_one(core)
```

Senior insight Specialization changes libraries, not the need for boundaries, reproducibility, and failure handling.

Go deeper → Python applications https://www.python.org/about/apps/

One primary source per need

Resource library and next 30 days

Use the official tutorial for language grounding, packaging.python.org for project structure, pytest for tests, typing.python.org for type-system depth, and FastAPI's tutorial for APIs. Exercism provides focused practice.

WHAT TO DO

- Week 1: rebuild the CLI without notes
- Week 2: package and test the pricing library
- Week 3: ship the API · Week 4: harden and document

```
BOOKMARKS = [
 'docs.python.org/3/',
 'packaging.python.org/',
 'docs.pytest.org/',
 'typing.python.org/',
 'fastapi.tiangolo.com/',
 'exercism.org/tracks/python',
]
```

Senior insight Your next milestone is not "finished Python." It is a deployed, tested service you can explain in a senior design review.

Go deeper → Official Python documentation https://docs.python.org/3/